

MetaDAO Futarchy

Smart Contract Security Assessment

July 2025

Prepared for:

MetaDAO

Prepared by:

Offside Labs

Sirius Xie

Gongyu Shi

Ripples Wen





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Insufficient Isolation of Locked LP Tokens Between Proposals	6
4.2	Missing Question Status Check Allows Unintended Behaviors	6
4.3	Missing Parameters Validation When Creating AMM, DAO, Proposal and Question Accounts	7
4.4	Ongoing Proposal Finalization Time May Be Affected by DAO Updates	8
4.5	Missing Validation of the Number of Question Outcomes in the initialize_proposal IX	9
4.6	DoS Risk from Frontrunning AMM Creation with Altered Parameters	10
4.7	Price Manipulation During Initial Liquidity Provision	11
4.8	Informational and Undetermined Issues	12
5	Disclaimer	16



1 About Offside Labs

Offside Labs stands as a pre-eminent security research team, comprising highly skilled hackers with top - tier talent from both academia and industry.

The team demonstrates extensive and diverse expertise in modern software systems, which encompasses yet are not restricted to *browsers, operating systems, IoT devices, and hypervisors*. Offside Labs is at the forefront of innovative domains such as *cryptocurrencies and blockchain technologies*. The team achieved notable accomplishments including the successful execution of remote jailbreaks on devices like the **iPhone** and **PlayStation 4**, as well as the identification and resolution of critical vulnerabilities within the **Tron Network**.

Offside Labs actively involves in and keeps contributing to the security community. The team was the winner and co-organizer for the *DEFCON CTF*, the most renowned CTF competition in Web2. The team also triumphed in the **Paradigm CTF 2023** in Web3. Meanwhile, the team has been conducting responsible disclosure of numerous vulnerabilities to leading technology companies, including *Apple, Google, and Microsoft*, safeguarding digital assets with an estimated value exceeding **\$300 million**.

During the transition to Web3, Offside Labs has attained remarkable success. The team has earned over **\$9 million** in bug bounties, and **three** of its innovative techniques were acknowledged as being among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



2 Executive Summary

Introduction

Offside Labs completed a security audit of *MetaDAO Futarchy* smart contracts, starting on June 5th, 2025, and concluding on June 30th, 2025.

Project Overview

The MetaDAO Futarchy project implements futarchy at scale within a DAO. In MetaDAO, proposals are not voted on directly, instead, participants trade on them.

Each proposal is associated with two markets: a “pass” market and a “fail” market. By depositing the quote token (or the base token), users receive an equal amount of market-specific quote tokens (or base tokens) for both the “pass” and “fail” markets. These tokens can only be traded within their respective markets, and such trading activity influences the price of the base token in each market. At the end of the proposal period, the decision, approval or rejection, is determined by comparing the price of the base token in the two markets. The market with the higher price indicates the winning outcome.

Audit Scope

The assessment scope contains mainly the smart contracts of the amm, autocrat and conditional_vault programs for the *MetaDAO Futarchy* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- MetaDAO Futarchy
 - Codebase: <https://github.com/metaDAOproject/futarchy>
 - Commit Hash: 1402b85df944789722f04ad7f17c7a647c760cf7

We listed the files we have audited below:

- MetaDAO Futarchy
 - programs/amm/src/events.rs
 - programs/amm/src/lib.rs
 - programs/amm/src/instructions/swap.rs
 - programs/amm/src/instructions/remove_liquidity.rs
 - programs/amm/src/instructions/create_amm.rs
 - programs/amm/src/instructions/crank_that_twap.rs
 - programs/amm/src/instructions/common.rs
 - programs/amm/src/instructions/add_liquidity.rs
 - programs/amm/src/state/amm.rs
 - programs/conditional_vault/src/events.rs
 - programs/conditional_vault/src/lib.rs
 - programs/conditional_vault/src/instructions/redeem_tokens.rs



- programs/conditional_vault/src/instructions/merge_tokens.rs
- programs/conditional_vault/src/instructions/split_tokens.rs
- programs/conditional_vault/src/instructions/initialize_conditional_vault.rs
- programs/conditional_vault/src/instructions/common.rs
- programs/conditional_vault/src/instructions/resolve_question.rs
- programs/conditional_vault/src/instructions/add_metadata_to_conditional_tokens.rs
- programs/conditional_vault/src/instructions/initialize_question.rs
- programs/conditional_vault/src/state/conditional_vault.rs
- programs/conditional_vault/src/state/question.rs
- programs/autocrat/src/events.rs
- programs/autocrat/src/lib.rs
- programs/autocrat/src/instructions/finalize_proposal.rs
- programs/autocrat/src/instructions/update_dao.rs
- programs/autocrat/src/instructions/initialize_dao.rs
- programs/autocrat/src/instructions/execute_proposal.rs
- programs/autocrat/src/instructions/initialize_proposal.rs
- programs/autocrat/src/state/proposal.rs
- programs/autocrat/src/state/dao.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 1 medium issue
- 6 low issues
- 9 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Insufficient Isolation of Locked LP Tokens Between Proposals	Medium	Fixed
02	Missing Question Status Check Allows Unintended Behaviors	Low	PartialFixed
03	Missing Parameters Validation When Creating AMM, DAO, Proposal and Question Accounts	Low	PartialFixed
04	Ongoing Proposal Finalization Time May Be Affected by DAO Updates	Low	Fixed
05	Missing Validation of the Number of Question Outcomes in the initialize_proposal IX	Low	Fixed
06	DoS Risk from Frontrunning AMM Creation with Altered Parameters	Low	Acknowledged
07	Price Manipulation During Initial Liquidity Provision	Low	Acknowledged
08	Missing Account Reloading in the split_tokens IX	Informational	Fixed
09	Unverified Owner of user_conditional_token_account in get_mints_and_user_token_accounts Function	Informational	Fixed
10	Missing Validation of the usdc_mint When Initialize DAO	Informational	Fixed
11	Inadequate Validation of conditional_token_mint When Adding the Metadata	Informational	Fixed
12	Loss of Accuracy Due to Per-Term Integer Division	Informational	Fixed
13	Inaccurate Account Data Size Calculation on Initialization	Informational	Fixed
14	Excessive Rent Transferred When Creating the Mint	Informational	Acknowledged
15	DoS Risk from Frontrunning Swap After Initial Liquidity Provision	Informational	Acknowledged
16	TWAP Is Still Accumulated When Proposal Minimum Duration is Reached	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Insufficient Isolation of Locked LP Tokens Between Proposals

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Design

Description

During the `initialize_proposal` process, the proposer locks a portion of LP tokens in a treasury-owned ATA account and these LP tokens are expected to be withdrawn during the `finalize_proposal` process. Although the LP tokens locked by different proposals are stored in separate ATA accounts, all of these accounts are owned by the treasury, which may not provide sufficient isolation between proposals.

Impact

If a malicious proposal is approved, the treasury may directly withdraw all LP tokens locked in other proposals of the DAO.

Recommendation

It is recommended to set the owner of the ATA account as the proposal itself.

Mitigation Review Log

Fixed in the commit `bcef04cbca3e8b05e5bba8a22cb83408e4566e4a`.

4.2 Missing Question Status Check Allows Unintended Behaviors

Severity: Low

Status: PartialFixed

Target: Smart Contract

Category: Logic Error

Description

In the `initialize_conditional_vault`, `split_token` and `merge_token` IXs, the status of the provided `question` is not checked. When the question is resolved, these IXs might still be invoked.



Impact

This omission may permit unintended behaviors that are inconsistent with the protocol's design. For instance, a user may still be able to split tokens after the proposal has ended, and subsequently perform a swap, and it may open up avenues for arbitrage.

Recommendation

It is recommended to add a check to ensure the question is not resolved for these instructions.

Mitigation Review Log

The check for the `initialize_conditional_vault` IX is added in the commit `f42d8bb1c6bd3f0a2248049261c2368e948adcbf`.

MetaDAO Team: We should do this for `initialize_conditional_vault` , not for `split_token` and `merge_token` .

4.3 Missing Parameters Validation When Creating AMM, DAO, Proposal and Question Accounts

Severity: Low

Status: PartialFixed

Target: Smart Contract

Category: Data Validation

Description

In the `initialize_dao` , `update_dao` , `create_amm` , `initialize_proposal` and `initialize_question` IXs, the passed-in parameters are not validated and are directly assigned to the corresponding fields in the accounts.

Impact

If certain parameters are set to unreasonable values, either deliberately or accidentally, it could result in unintended behavior. Here are some examples:

1. Regarding the DAO parameters, if `slots_per_proposal` is smaller than `twap_start_delay_slots` , a proposal may be finalized before the AMM pool has started recording TWAP. Additionally, `twap_start_delay_slots` should be validated to ensure that TWAP recording only begins after initial liquidity has been added to the pool.
2. The AMM and DAO both include the fields `twap_initial_observation` and `twap_max_observation_change_per_update` . The former represents the initial observation value, while the latter defines the maximum absolute change allowed per update in order to prevent attackers from manipulating the market and causing drastic price



swings that would affect the TWAP calculation. However, if the latter is significantly larger than the former, for example, if `twap_initial_observation` is 10000 and `twap_max_observation_change_per_update` is 100000, then the restriction becomes ineffective and provides no protection.

3. When creating a proposal, the parameters `pass_lp_tokens_to_lock` and `fail_lp_tokens_to_lock` are not validated. Due to this omission, the proposer can supply different amounts of quote tokens (USDC) in the initial liquidity. This issue can be compounded with the previously mentioned price manipulation vulnerability, as it allows the amounts of both base and quote tokens in the initial pass and fail AMM pools to differ. This discrepancy can potentially compromise the fairness of the proposal.
4. The `oracle` of the question is not validated if the question is associated with a prediction market. The oracle should be an authorized party, as it is responsible for resolving the question.

Recommendation

It is recommended to add the following checks:

1. For DAO accounts, `twap_start_delay_slots` should be greater than the number of slots corresponding to 5 minutes, since a necessary condition for a proposal to be successfully created is that its associated AMM pools must have been created less than 5 minutes ago; `slots_per_proposal` should be always greater than `twap_start_delay_slots`.
2. In both DAO and AMM accounts, the ratio of `twap_max_observation_change_per_update` to `twap_initial_observation` should be restricted to ensure meaningful limits on TWAP updates.
3. For proposal accounts, the value of `pass_lp_tokens_to_lock` could be equal to `fail_lp_tokens_to_lock`.
4. For question accounts, its oracle should be validated.

Mitigation Review Log

The check for `slots_per_proposal` is added in the commit `0bbff3e6a28d01a04f301fe83230748cf12aacc3`.

MetaDAO Team: Others are fine and not logical bugs.

4.4 Ongoing Proposal Finalization Time May Be Affected by DAO Updates

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error



Description

In the `finalize_proposal` instruction, there is a check to ensure that the proposal has been live for the required duration. However, this validation uses the value stored in the DAO account (`self.dao.slots_per_proposal`), rather than the value recorded in the proposal account itself (`self.proposal.duration_in_slots`).

```
57 require!(  
58     clock.slot >= self.proposal.slot_enqueued + self.dao.slots_per_proposal,  
59     AutocratError::ProposalTooYoung  
60 );
```

[programs/autocrat/src/instructions/finalize_proposal.rs#L57-L60](#)

Impact

If a proposal is currently live and another proposal that updates the DAO's `slots_per_proposal` is passed, the ongoing proposal's finalization timing may be unexpectedly changed.

Recommendation

It is recommended to use `self.proposal.duration_in_slots` in the validation above.

Mitigation Review Log

Fixed in the commit `2e45a2dbeb2d2c9e54306ee413910006328057dc`.

4.5 Missing Validation of the Number of Question Outcomes in the `initialize_proposal` IX

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

In the `initialize_proposal` IX, it is assumed that the provided `question` account has exactly two outcomes, but this is not validated.

```
29 #[account(  
30     constraint = question.oracle == proposal.key()  
31 )]  
32 pub question: Box<Account<'info', Question>>,
```

[programs/autocrat/src/instructions/initialize_proposal.rs#L29-L32](#)



Impact

Suppose a question with three or more outcomes is provided and the proposal is initialized successfully, then the proposal can never be finalized, due to the failure of the check for `args.payout_numerators.len()`. As a result, the locked LP tokens of the proposer cannot be withdrawn.

Recommendation

It is recommended to add a check to ensure the `question` has exactly two outcomes.

Mitigation Review Log

Fixed in the commit 81908e6d52e788238a8b4874a10a5eaa576c3e09.

4.6 DoS Risk from Frontrunning AMM Creation with Altered Parameters

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: DoS Risk

Description

During the initialization of a proposal in the `initialize_proposal` IX, certain fields of the provided `amm` account are validated as follows

```
110         require_eq!(  
111             amm.oracle.start_delay_slots,  
112             self.dao.twap_start_delay_slots,  
113             AutocratError::InvalidStartDelaySlots  
114         );
```

[programs/autocrat/src/instructions/initialize_proposal.rs#L110-L114](#)

These fields are initially set when the `amm` account is created via the `creata_amm` IX, with the expectation that it is created by the proposer or other legitimate users. However, once the two conditional vaults are created, a malicious user might front-run the `create_amm` transaction to create the same `amm` account with altered oracle parameters.

Impact

The proposal cannot be initialized because the validation described above may consistently fail, resulting in a DoS vulnerability.



Recommendation

It is recommended to refer to the proposal initialization and introduce a nonce when deriving the `amm` address.

Mitigation Review Log

MetaDAO Team: Jito Bundles are utilized to ensure the ordered and atomic execution of these transactions.

4.7 Price Manipulation During Initial Liquidity Provision

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

Description

```
91 // equivalent to $100 if quote is USDC, here for rounding
92 require_gte!(quote_amount, 100000000, AmmError::InsufficientQuoteAmount);
93
94 let base_amount = max_base_amount;
95
96 let initial_lp_tokens = quote_amount;
97
98 (initial_lp_tokens, base_amount)
```

[programs/amm/src/instructions/add_liquidity.rs#L91-L98](#)

During the initial liquidity provision to the AMM pool, only the deposited quote token amount is validated, while no further checks are performed on the base token amount (there is a basic check to ensure it is greater than 0). As a result, a user may supply base tokens in a ratio that deviates from the fair market value.

Impact

The proposer can add the same amount of pass quote tokens and fail quote tokens to the pass AMM and fail AMM respectively, while supplying different amounts of pass base tokens and fail base tokens. This results in differing initial TWAP prices between the two pools, which may, under certain conditions, affect the aggregator's calculations. Consequently, the fairness of the proposal itself may be compromised.



Recommendation

It is recommended to perform additional validation on the deposited base token amount during the first `add_liquidity` operation in a new AMM.

One possible solution is that the base amount could be calculated based on the `quote_amount` and the initial price `initial_observation`. Although `initial_observation` may also deviate from the real-time market price, at least the deviation would be within a relatively reasonable range.

Mitigation Review Log

MetaDAO Team: It is reasonable that the two AMMs have different ratios, their fairs might be different.

4.8 Informational and Undetermined Issues

Missing Account Reloading in the `split_tokens` IX

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

In the `split_tokens` IX, the `user_underlying_token_account` is not reloaded after the token transfer. Consequently, the emitted event contains the incorrect value.

Unverified Owner of `user_conditional_token_account` in `get_mints_and_user_token_accounts` Function

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In the `get_mints_and_user_token_accounts` function, the `user_conditional_token_accounts` will be extracted from the remaining accounts. But the owners of these token accounts are not validated, they may differ from the passed-in `authority` account.

Missing Validation of the `usdc_mint` When Initialize DAO

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In the `initialize_dao` IX, the provided `usdc_mint` account is not fully validated. If the mint is strictly required to be USDC, it is recommended to explicitly check whether `usdc_mint` equals the expected mint address. Otherwise, it is recommended to rename this field to avoid confusion.



Inadequate Validation of conditional_token_mint When Adding the Metadata

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In the `add_metadata_to_conditional_tokens` IX, the authority of the passed-in `conditional_token_mint` is checked. But it might not be the mint account of any conditional tokens in that vault. It is recommended to add this validation.

Loss of Accuracy Due to Per-Term Integer Division

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Precision Issue

```
56     total_redeemable += ((user_conditional_token_account.amount as u128
57         * question.payout_numerators[payout_index] as u128)
58         / question.payout_denominator as u128) as u64;
```

[programs/conditional_vault/src/instructions/redeem_tokens.rs#L56-L58](#)

```
45     conditional_token_supplies
46         .iter()
47         .enumerate()
48         .map(|(i, supply)| {
49             *supply as u128 * question.payout_numerators[i] as u128
50             / question.payout_denominator as u128
51         })
52         .sum::<u128>() as u64
```

[programs/conditional_vault/src/state/conditional_vault.rs#L45-L52](#)

In the examples above, expressions such as

$$s = \lfloor \frac{a_1}{b} \rfloor + \lfloor \frac{a_2}{b} \rfloor + \dots + \lfloor \frac{a_n}{b} \rfloor$$

are calculated. However, performing individual floor division for each item may result in greater rounding loss.

It is recommended to sum all numerators first and then divide by the denominator. Note that this issue does not affect the proposals and prediction markets, as there is only one non-zero numerator.

Inaccurate Account Data Size Calculation on Initialization

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA



During the initialization of the `Amm`, `Dao`, and `ConditionalVault` accounts, the implementation utilizes `std::mem::size_of` to determine the account data size. However, this approach does not always yield the correct size after serialization using `BorshSerialize`.

Additionally, the space calculation for the `Question` account is defined as `8 + 32 + 32 + 1 + 4 + (args.num_outcomes as usize * 4) + 4`. However, the `+ 1` component is redundant and does not correspond to any field within the struct.

It is strongly recommended to use the `InitSpace` macro to calculate account space requirements. The `InitSpace` macro provides a more accurate calculation based on the actual serialized structure and enhances maintainability by minimizing manual errors in space estimation.

Excessive Rent Transferred When Creating the Mint

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

In the `initialize_conditional_vault` IX, when creating the mint accounts for the condition tokens, the default rent for a valid mint account is transferred. However, if the lamports balance of the target is not zero, fewer lamports are needed.

It is suggested to calculate the transferred lamports amount based on the `mint_lamports` and the balance of the target account.

```
46     let mint_lamports = Rent::get()?.minimum_balance(Mint::LEN);
47     for i in 0..expected_num_conditional_tokens {
48         ...
49         system_program::transfer(cpi_ctx, mint_lamports)?;
```

[programs/conditional_vault/src/instructions/initialize_conditional_vault.rs#L46-L49](#)

DoS Risk from Frontrunning Swap After Initial Liquidity Provision

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: DoS Risk

In the `initialize_proposal` instruction, the system ensures that the proposer holds sufficient liquidity of both base and quote tokens in the two AMM pools.

```
let (pass_base_liquidity, pass_quote_liquidity) =
    pass_amm.get_base_and_quote_withdrawable(pass_lp_tokens_to_lock,
    pass_lp_mint.supply);
...
for base_liquidity in [pass_base_liquidity, fail_base_liquidity] {
    require_gte!(
```



```
    base_liquidity,  
    dao.min_base_futarchic_liquidity,  
    AutocratError::InsufficientLpTokenLock  
);  
}
```

[programs/autocrat/src/instructions/initialize_proposal.rs#L164-L183](#)

However, if the initial liquidity provision and proposal creation are not atomically, a malicious user can frontrun the proposal creation by performing a swap. As a result, the proposer's liquidity might fall below the required minimum after the swap, potentially causing a DoS

TWAP Is Still Accumulated When Proposal Minimum Duration is Reached

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

When a proposal satisfies the required duration, the AMM pools should stop recording TWAP, to prevent scenarios where delayed finalization allows unexpected price changes to affect the proposal result. It is recommended to add a check on the current slot to determine the stop time whenever updating the TWAP.



5 Disclaimer

This report reflects the security status of the project as of the date of the audit. It is intended solely for informational purposes and should not be used as investment advice. Despite carrying out a comprehensive review and analysis of the relevant smart contracts, it is important to note that Offside Labs' services do not encompass an exhaustive security assessment. The primary objective of the audit is to identify potential security vulnerabilities to the best of the team's ability; however, this audit does not guarantee that the project is entirely immune to future risks.

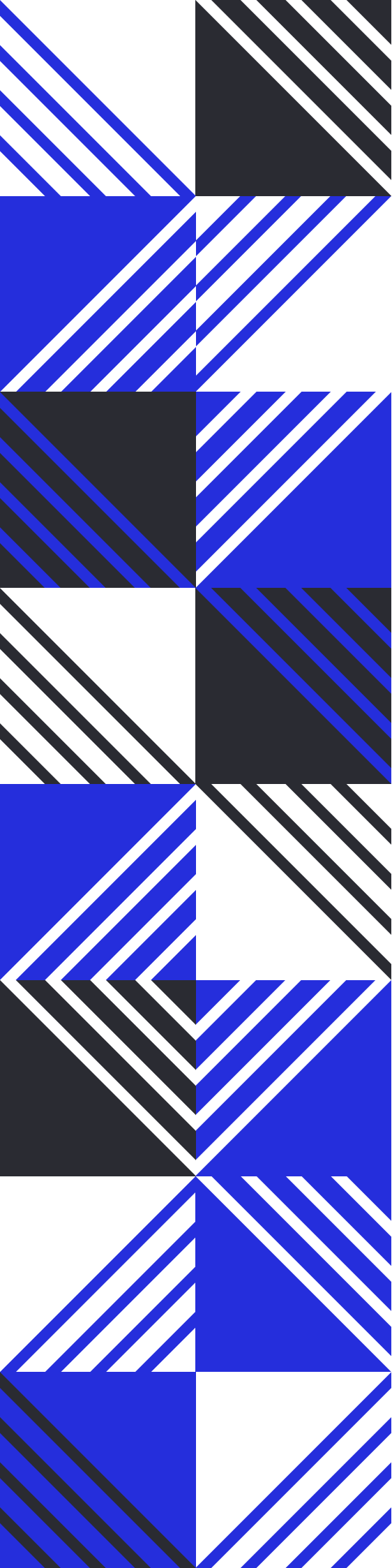
Offside Labs disclaims any liability for losses or damages resulting from the use of this report or from any future security breaches. The team strongly recommends that clients undertake multiple independent audits and implement a public bug bounty program to enhance the security of their smart contracts.

The audit is limited to the specific areas defined in Offside Labs' engagement and does not cover all potential risks or vulnerabilities. Security is an ongoing process, regular audits and monitoring are advised.

Please note: Offside Labs is not responsible for security issues stemming from developer errors or misconfigurations during contract deployment and does not assume liability for centralized governance risks within the project. The team is not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By utilizing this report, the client acknowledges the inherent limitations of the audit process and agrees that the firm shall not be held liable for any incidents that may occur after the completion of this audit.

This report should be considered null and void in case of any alteration.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs